

Implementation Plan - Driver Drowsiness Detection with Explainable AI (XAI)

Build a complete, end-to-end Deep Learning system for **Driver Drowsiness Detection** utilizing architectures such as **VGG16**, **MobileNetV2**, and **Custom CNNs**, fully integrated with **Explainable AI (XAI)** techniques like **Grad-CAM** and feature visualizers. The system includes dataset pre-processing, model training, XAI evaluation, and an interactive real-time dashboard.

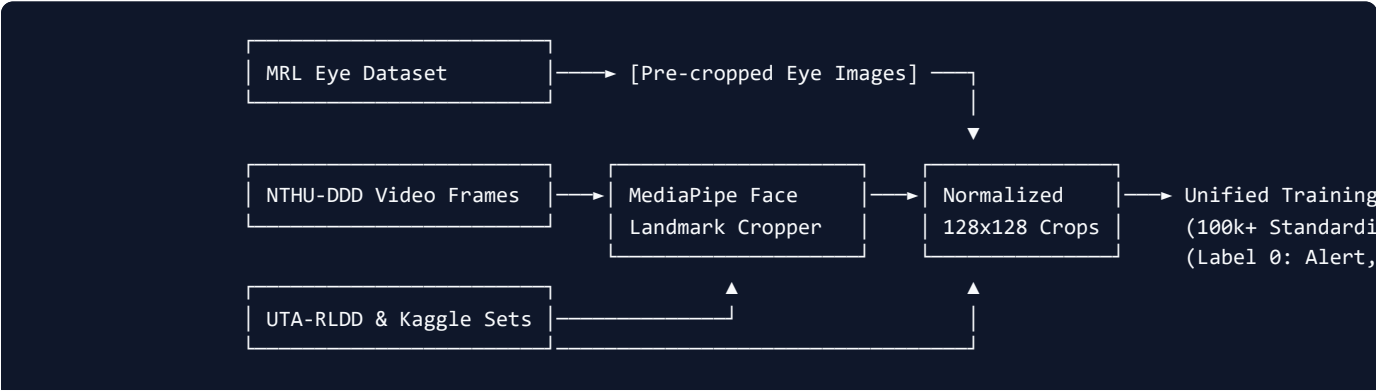
1. Dataset Recommendations & Strategy

To train a robust drowsiness detection model, we recommend a multi-tier dataset strategy depending on project scope:

Dataset Name	Primary Focus	Best Used For	Format / Size	Availability
MRL Eye Dataset <i>(Recommended Initial)</i>	Eye Open vs. Closed under varying lighting, glasses, & head poses	Eye-state classification baseline	~84,000 cropped images	Free / Kaggle / Direct Download
Drowsiness Dataset (Kaggle)	4 Classes: Closed Eyes, Open Eyes, Yawn, No Yawn	Combined eye & yawning classification	~3,000 cropped face/mouth/eye images	Free on Kaggle
NTHU Driver Drowsiness Dataset (NTHU-DDD)	Video sequences: normal, yawning, slow blink, nodding	Temporal & video-sequence evaluation	36 IR video clips with full annotation	Academic Request / Download
UTA-RLDD	Real-life micro-expressions & multi-stage drowsiness	Advanced multi-class fatigue modeling	~30 hours RGB video (60 subjects)	Academic Request

Unified Dataset Fusion Strategy (Combining MRL, Kaggle, NTHU & UTA-RLDD):

Combining multiple datasets is a **highly effective technique** to increase generalization across different drivers, lighting, and camera angles. However, because some datasets contain cropped eyes (MRL) while others contain full driver video frames (NTHU, UTA-RLDD), you should follow this **Unified Preprocessing Pipeline**:



1. Standardize Label Mapping:

2. Map Open Eyes , No Yawn , Normal Driving $\rightarrow$ Label 0 (Alert)

3. Map Closed Eyes , Yawn , Slow Blink , Drowsy $\rightarrow$ Label 1 (Drowsy)

4. Automated Facial Landmark ROI Extraction:

5. For full face datasets (NTHU, UTA-RLDD, Kaggle full frames), run **MediaPipe Face Mesh** to detect eye bounding boxes and crop left/right eyes into normalized 128×128 resolution images.
 6. Combine these cropped eyes with the pre-cropped **MRL Eye Dataset**.
 7. **Stratified Split:**
 8. Perform subject-aware or stratified train/val/test splitting (80% Train, 10% Val, 10% Test) to ensure balanced class distributions and prevent data leakage across video frames.
-

2. Model Architectures to Compare

We will implement a unified modular model factory supporting an exhaustive benchmark suite of deep learning architectures across **5 model paradigms**:

Paradigm A: Classic Heavyweight Convolutional Neural Networks (CNNs)

1. **VGG16 / VGG19 (Transfer Learning):**
2. *Parameters:* ~138M (or ~15M with dynamic global average pooling classifier head).
3. *Target Layer for Grad-CAM:* `features.28 ( block5_conv3 )`.
4. *Best For:* Rich baseline feature maps, deep representational capacity, and visually clean Grad-CAM heatmaps.
5. **Custom 5-Layer CNN (Baseline):**
6. *Parameters:* ~0.5M - 1.2M.
7. *Target Layer for Grad-CAM:* Final Conv layer `conv5`.
8. *Best For:* Lightweight baseline trained from scratch without pre-trained ImageNet bias.

Paradigm B: Residual & Dense Architectures

1. **ResNet18 / ResNet50:**
2. *Parameters:* ~11.7M (ResNet18) / ~25.6M (ResNet50).
3. *Target Layer for Grad-CAM:* `layer4[-1]` (final bottleneck/residual block).
4. *Best For:* Deep residual learning, preventing vanishing gradients, and fast convergence with strong accuracy.
5. **ResNeXt50 (32x4d):**
6. *Parameters:* ~25.0M.
7. *Target Layer for Grad-CAM:* `layer4[-1]`.
8. *Best For:* Grouped multi-branch convolutions capturing diverse micro-facial features (eye flutter, eyelid crease).
9. **DenseNet121:**
10. *Parameters:* ~8.0M.
11. *Target Layer for Grad-CAM:* `features.denseblock4`.
12. *Best For:* Feature reuse through dense connections, yielding rich gradient flow with fewer parameters.

Paradigm C: Lightweight Edge & Embedded Mobile Models (Real-Time In-Vehicle Devices)

1. **MobileNetV2 / MobileNetV3-Small & Large:**
2. *Parameters:* ~3.5M (V2) / ~2.5M (V3-Small).
3. *Target Layer for Grad-CAM:* `features[-1]`.
4. *Best For:* Depthwise separable convolutions optimized for high FPS (30+ FPS) on Raspberry Pi / Android edge hardware.
5. **ShuffleNetV2 (1.0x):**
6. *Parameters:* ~2.3M.

7. *Target Layer for Grad-CAM:* `conv5` .
8. *Best For:* Channel shuffle operations maximizing computational throughput per FLOP.
9. **SqueezeNet1_1:**
10. *Parameters:* ~1.2M.
11. *Target Layer for Grad-CAM:* `features.12` (Fire module 9).
12. *Best For:* Extremely small model binary footprint (~4.8 MB), ideal for microcontrollers.

Paradigm D: Modern Neural Architecture Search (NAS) & Pure ConvNeXt

1. **EfficientNet-B0 / EfficientNet-B2:**
2. *Parameters:* ~5.3M (B0) / ~9.2M (B2).
3. *Target Layer for Grad-CAM:* `features[-1]` .
4. *Best For:* AutoML compound scaling (depth, width, resolution) balancing top-tier accuracy and computational cost.
5. **ConvNeXt-Tiny / ConvNeXt-Small:**
 - *Parameters:* ~28M (Tiny).
 - *Target Layer for Grad-CAM:* `features.7` (Stage 4 depthwise layer).
 - *Best For:* Modernized pure convolutional networks incorporating Vision Transformer design choices (7x7 depthwise kernels, inverted bottlenecks).

Paradigm E: Vision Transformers (ViT) & Attention Models

1. **Vision Transformer (ViT-Tiny / ViT-Base):**
 - *Parameters:* ~5.7M (ViT-Tiny patch16) / ~86M (ViT-Base).
 - *XAI Method:* Attention Rollout & Attention Maps (instead of Grad-CAM).
 - *Best For:* Global self-attention capturing interactions across face, eyes, and mouth simultaneously.
 2. **Swin Transformer (Swin-Tiny):**
 - *Parameters:* ~28M.
 - *XAI Method:* Layer-wise Attention / Grad-CAM for Swin.
 - *Best For:* Hierarchical shifted windows providing strong multi-scale feature localization.
-

Comparative Evaluation Metrics Table (To be benchmarked):

Model Name	Paradigm	Approx. Params	Expected CPU FPS	Top-1 Accuracy	Grad-CAM Clarity	Edge Suitable?
Custom CNN	Baseline CNN	~0.8M	60+ FPS	Baseline	High	Yes (Micro)
VGG16	Heavy CNN	~15M (Modified)	~15 FPS	High	Extremely High	No (Server/GPU)
ResNet18	Residual	~11.7M	40+ FPS	High	High	Yes
ResNet50	Residual	~25.6M	25 FPS	Very High	High	Moderate
MobileNetV2	Mobile Edge	~3.5M	50+ FPS	High	Moderate-High	Yes (Best Edge)
MobileNetV3-Small	Mobile Edge	~2.5M	60+ FPS	Moderate-High	Moderate	Yes
EfficientNet-B0	NAS / Compound	~5.3M	35 FPS	Very High	High	Yes
ConvNeXt-Tiny	Modern Conv	~28M	20 FPS	Very High	Very High	Moderate
ViT-Tiny	Transformer	~5.7M	25 FPS	High	Attention Rollout	Moderate

3. Explainable AI (XAI) Suite

To provide transparent decision-making (answering *why* the model flagged a driver as drowsy):

- **Grad-CAM (Gradient-weighted Class Activation Mapping):**
 - Generates visual heatmaps over input images by computing gradients of the target class score with respect to feature maps of the final convolutional layer.
 - Overlays heatmaps on eye lids, pupil region, or mouth to show exactly which facial regions influenced the decision.
- **Grad-CAM++ / Layer-CAM:**
 - Provides finer resolution for multiple salient objects (e.g., both eyes and mouth simultaneously).
- **Metric Analytics:**
 - Computes Eye Aspect Ratio (EAR) & Mouth Aspect Ratio (MAR) metrics side-by-side with model predictions to validate model visual reasoning against geometric benchmarks.

4. User Review Required

[!IMPORTANT]

Framework & Dataset Preference:

1. **Framework:** PyTorch (recommended for clean Grad-CAM hooks and torch-cam) or TensorFlow/Keras? PyTorch will be the default unless requested otherwise.
2. **Dataset Choice:** Should we create automated download & setup scripts for the **MRL Eye Dataset / Kaggle Drowsiness dataset**, or do you have a local dataset ready?
3. **UI Dashboard:** Should we build a **Streamlit Web Application** for interactive webcam stream + XAI heatmaps, or a PySide6/OpenCV desktop UI? Streamlit is recommended for web-based accessibility.

5. Proposed Project Directory Structure

```
drowsiness_detection/
├── data/
│   ├── download_dataset.py    # Automated dataset fetching & extraction
│   └── dataset_loader.py      # PyTorch Dataset/DataLoader with Augmentations
├── models/
│   ├── model_factory.py       # Factory to build Custom CNN, VGG, ResNet, MobileNet, EfficientNet, ViT
│   ├── custom_cnn.py          # Custom baseline 5-layer ConvNet
│   ├── vgg_model.py           # VGG16/VGG19 transfer learning model
│   ├── resnet_model.py        # ResNet18/ResNet50 model
│   ├── mobilenet_model.py     # MobileNetV2/V3 transfer learning model
│   ├── efficientnet_model.py  # EfficientNet-B0/B2 model
│   └── vit_model.py           # Vision Transformer (ViT-Tiny) model
├── xai/
│   ├── grad_cam.py            # Grad-CAM, Grad-CAM++, and Layer-CAM engine
│   ├── vit_attention.py       # Attention rollout visualizer for Transformers
│   └── visualizer.py          # Heatmap overlay & multi-model comparison plotter
├── utils/
│   ├── face_mesh.py           # MediaPipe landmark extraction (EAR & MAR computation)
│   └── metrics.py             # Evaluation metrics (Accuracy, F1, Confusion Matrix, FPS latency)
├── train.py                   # Training pipeline with validation, model benchmark, & XAI checkpoints
├── predict.py                 # Single image / video batch inference + Grad-CAM rendering
├── app.py                     # Streamlit interactive Web Application & live video stream
├── requirements.txt           # Project dependencies (torch, torchvision, opencv, streamlit, mediapipe, timm)
└── README.md                  # System overview and quickstart guide
```

6. Proposed Implementation Roadmap

Step 1: Requirements & Data Setup

- Define `requirements.txt` with PyTorch, Torchvision, OpenCV, MediaPipe, Streamlit, Matplotlib, Albumentations, NumPy, and `timm` (PyTorch Image Models).
- Script automatic fetching/synthetic generator or MRL dataset preparation into `data/train` and `data/val`.

Step 2: Model Factory & Multi-Architecture Benchmark (`models/` , `train.py`)

- Implement `model_factory.py` capable of instantiating all candidate architectures:
- Baseline CNN, VGG16/19, ResNet18/50, MobileNetV2/V3, EfficientNet-B0/B2, ConvNeXt-Tiny, ViT-Tiny.
- Implement training loop with CrossEntropyLoss, AdamW optimizer, Cosine Annealing LR Scheduler, and Early Stopping.
- Benchmark and log: Accuracy, Precision, Recall, F1-Score, Parameter count, and Inference Latency (FPS on CPU/GPU).

Step 3: Explainable AI Engine (`xai/`)

- Implement forward/backward hook-based Grad-CAM engine supporting ConvNets (VGG, ResNet, MobileNet, EfficientNet, ConvNeXt).
- Implement Attention Rollout visualization for Transformer models (ViT).
- Build visualization pipeline to fuse high-contrast colormap (e.g. `JET` or `TURBO`) over original input images.
- Generate multi-model side-by-side diagnostic heatmaps for direct explainability comparison.

Step 4: Geometric Metrics & MediaPipe Integration (`utils/face_mesh.py`)

- Calculate Eye Aspect Ratio (EAR) using facial landmarks:
$$\text{EAR} = \frac{|p_2 - p_6| + |p_3 - p_5|}{2 |p_1 - p_4|}$$
- Combine CNN prediction probabilities with EAR thresholding for hybrid verification.

Step 5: Interactive Streamlit Dashboard (`app.py`)

- Build a web interface offering:
 - **Upload Image/Video:** Test model predictions and view real-time XAI Grad-CAM overlay.
 - **Live Webcam Stream:** Real-time drowsiness monitoring with audio alert triggers (PERCLOS calculation).
 - **Model Inspection Tab:** Compare accuracy vs latency across VGG16, MobileNetV2, and Custom CNN.
-

7. Verification Plan

Automated Verification

- Run model initialization tests (`pytest` or verification script) to confirm forward pass shapes across VGG16, MobileNetV2, and Custom CNN.
- Run Grad-CAM module sanity test ensuring heatmap array dimensions match input image specs (`C, H, W`) .

Manual Verification

- Train model for baseline epoch on dataset sample.
- Test prediction on sample open/closed eye images and verify that Grad-CAM heatmaps focus on eye/eyelid regions.
- Launch Streamlit dashboard on `http://localhost:8501` to test webcam/file upload feed and interactive controls.